Chapter 1. Introduction to CUDA

1.1. Introduction

1.1.1. The Graphics Processing Unit

Born as a special-purpose processor for 3D graphics, the *Graphics Processing Unit* (GPU) started out as fixed-function hardware to accelerate parallel operations in real-time 3D rendering. Over successive generations, GPUs became more programmable. By 2003, some stages of the graphics pipeline became fully programmable, running custom code in parallel for each component of a 3D scene or an image.

In 2006, NVIDIA introduced the *Compute Unified Device Architecture* (CUDA) to enable any computational workload to use the throughput capability of GPUs independent of graphics APIs.

Since then, CUDA and GPU computing have been used to accelerate computational workloads of nearly every type, from scientific simulations such as fluid dynamics or energy transport to business applications like databases and analytics. Moreover, the capability and programmability of GPUs has been foundational to the advancement of new algorithms and technologies ranging from image classification to generative artificial intelligence such as diffusion or large language models.

1.1.2. The Benefits of Using GPUs

A GPU provides much higher instruction throughput and memory bandwidth than a CPU within a similar price and power envelope. Many applications leverage these capabilities to run significantly faster on the GPU than on the CPU (see GPU Applications). Other computing devices, like FPGAs, are also very energy efficient, but offer much less programming flexibility than GPUs.

GPUs and CPUs are designed with different goals in mind. While a CPU is designed to excel at executing a serial sequence of operations (called a thread) as fast as possible and can execute a few tens of these threads in parallel, a GPU is designed to excel at executing thousands of threads in parallel, trading off lower single-thread performance to achieve much greater total throughput.

GPUs are specialized for highly parallel computations and devote more transistors to data processing units, while CPUs dedicate more transistors to data caching and flow control. Figure 1 shows an example distribution of chip resources for a CPU versus a GPU.

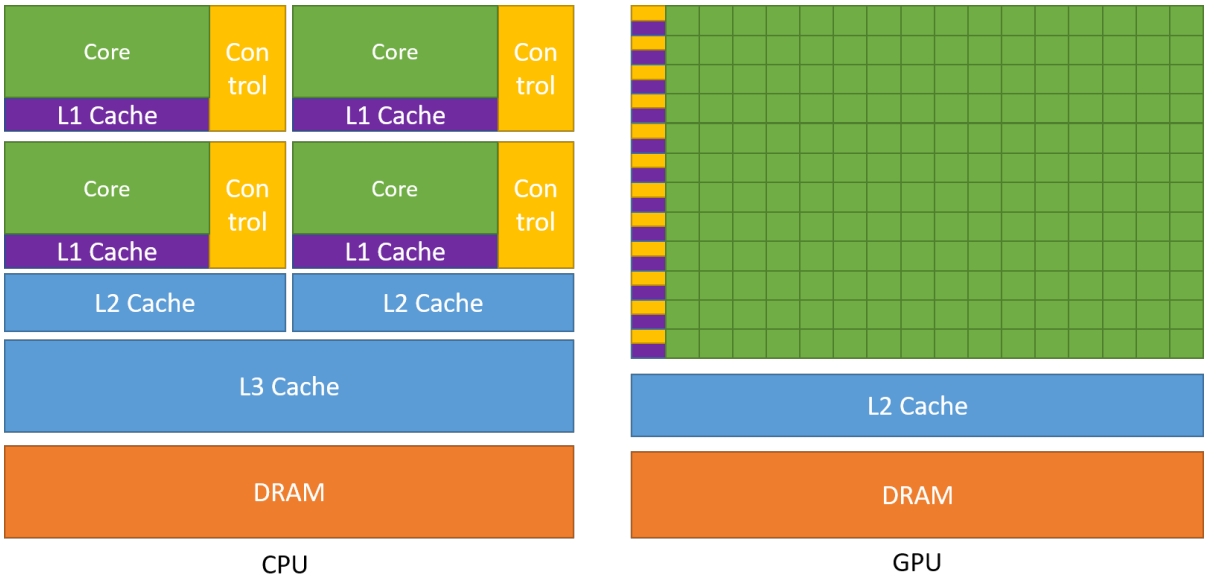

Figure 1: The GPU Devotes More Transistors to Data Processing

1.1.3. Getting Started Quickly

There are many ways to leverage the compute power provided by GPUs. This guide covers programming for the CUDA GPU platform in high-level languages such as C++. However, there are many ways to utilize GPUs in applications that do not require directly writing GPU code.

An ever-growing collection of algorithms and routines from a variety of domains is available through specialized libraries. When a library has already been implemented—especially those provided by NVIDIA—using it is often more productive and performant than reimplementing algorithms from scratch. Libraries like cuBLAS, cuFFT, cuDNN, and CUTLASS are just a few examples of libraries that help developers avoid reimplementing well-established algorithms. These libraries have the added benefit of being optimized for each GPU architecture, providing an ideal mix of productivity, performance, and portability.

There are also frameworks, particularly those used for artificial intelligence, that provide GPU-accelerated building blocks. Many of these frameworks achieve their acceleration by leveraging the GPU-accelerated libraries mentioned above.

Additionally, domain-specific languages (DSLs) such as NVIDIA's Warp or OpenAI's Triton compile to run directly on the CUDA platform. This provides an even higher-level method of programming GPUs than the high-level languages covered in this guide.

The NVIDIA Accelerated Computing Hub contains resources, examples, and tutorials to teach GPU and CUDA computing.

1.2. Programming Model

This chapter introduces the CUDA programming model at a high level and separate from any language. The terminology and concepts introduced here apply to CUDA in any supported programming language. Later chapters will illustrate these concepts in C++.

1.2.1. Heterogeneous Systems

The CUDA programming model assumes a heterogeneous computing system, which means a system that includes both GPUs and CPUs. The CPU and the memory directly connected to it are called the *host* and *host memory*, respectively. A GPU and the memory directly connected to it are referred to as the *device* and *device memory*, respectively. In some system-on-chip (SoC) systems, these may be part of a single package. In larger systems, there may be multiple CPUs or GPUs.

CUDA applications execute some part of their code on the GPU, but applications always start execution on the CPU. The host code, which is the code that runs on the CPU, can use CUDA APIs to copy data between the host memory and device memory, start code executing on the GPU, and wait for data copies or GPU code to complete. The CPU and GPU can both be executing code simultaneously, and best performance is usually found by maximizing utilization of both CPUs and GPUs.

The code an application executes on the GPU is referred to as *device code*, and a function that is invoked for execution on the GPU is, for historical reasons, called a *kernel*. The act of starting a kernel running is called *launching* the kernel. A kernel launch can be thought of as starting many threads executing the kernel code in parallel on the GPU. GPU threads operate similarly to threads on CPUs, though there are some differences important to both correctness and performance that will be covered in later sections (see Section 3.2.2.1.1).

1.2.2. GPU Hardware Model

Like any programming model, CUDA relies on a conceptual model of the underlying hardware. For the purposes of CUDA programming, the GPU can be considered to be a collection of *Streaming Multiprocessors* (SMs) which are organized into groups called *Graphics Processing Clusters* (GPCs). Each SM contains a local register file, a unified data cache, and a number of functional units that perform computations. The unified data cache provides the physical resources for *shared memory* and L1 cache. The allocation of the unified data cache to L1 and shared memory can be configured at runtime. The sizes of different types of memory and the number of functional units within an SM can vary across GPU architectures.

Note

The actual hardware layout of a GPU or the way it physically carries out the execution of the programming model may vary. These differences do not affect correctness of software written using the CUDA programming model.

1.2.2.1 Thread Blocks and Grids

When an application launches a kernel, it does so with many threads, often millions of threads. These threads are organized into blocks. A block of threads is referred to, perhaps unsurprisingly, as a *thread block*. Thread blocks are organized into a *grid*. All the thread blocks in a grid have the same size and dimensions. Figure 3 shows an illustration of a grid of thread blocks.

Thread blocks and grids may be 1, 2, or 3 dimensional. These dimensions can simplify mapping of individual threads to units of work or data items.

When a kernel is launched, it is launched using a specific *execution configuration* which specifies the grid and thread block dimensions. The execution configuration may also include optional parameters such as cluster size, stream, and SM configuration settings, which will be introduced in later sections.

Using built-in variables, each thread executing the kernel can determine its location within its containing block and the location of its block within the containing grid. A thread can also use these

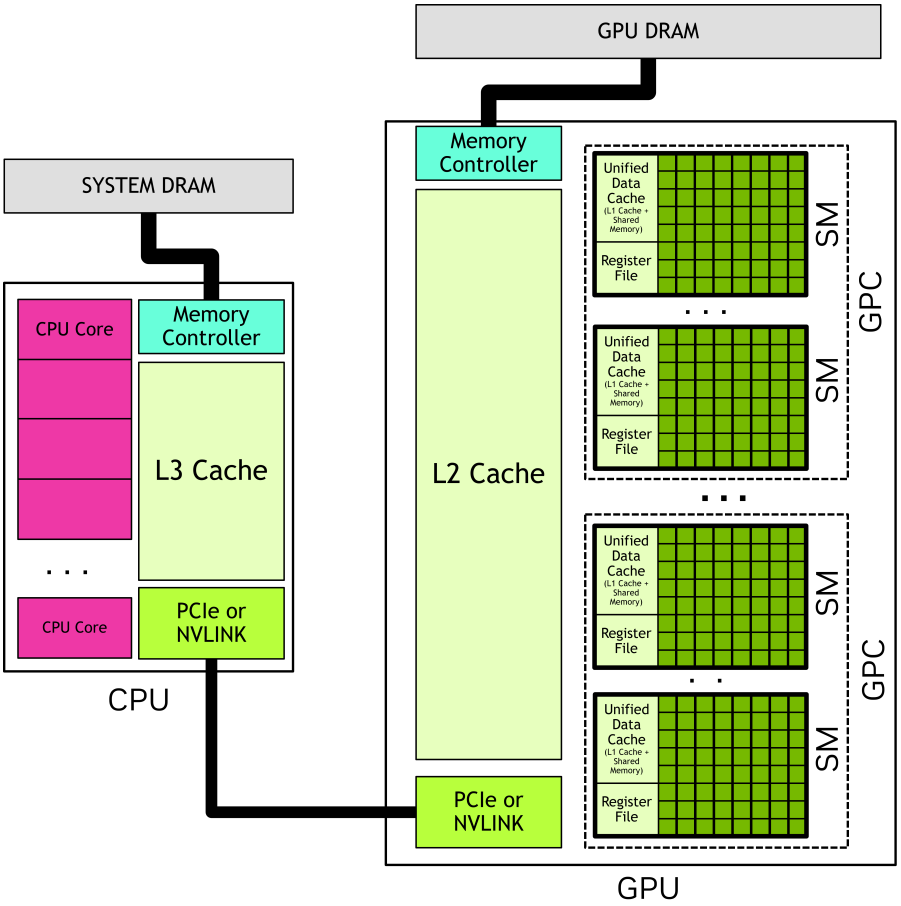

Figure 2: A GPU has many streaming multiprocessors (SMs), each of which contains many functional units. Graphics processing clusters (GPCs) are collections of SMs. A GPU is a set of GPCs connected to the GPU memory. A CPU typically has several cores and a memory controller which connects to the system memory. A CPU and a GPU are connected by an interconnect such as PCIe or NVLINK.

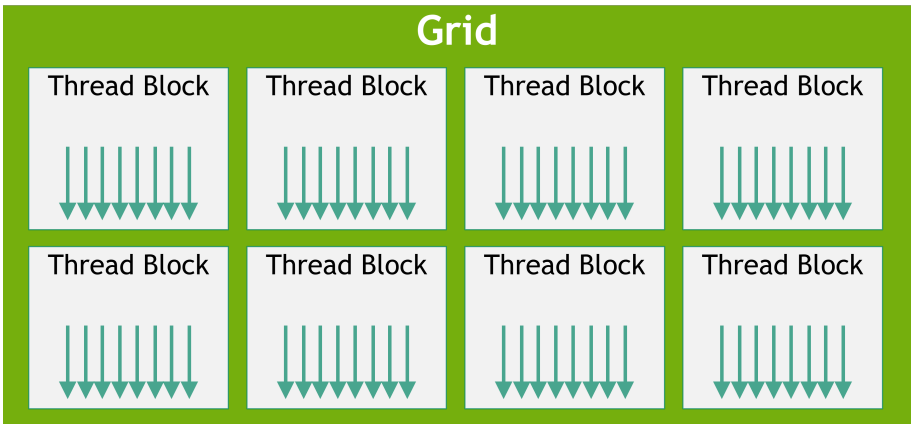

Figure 3: Grid of Thread Blocks. Each arrow represents a thread (the number of arrows is not representative of actual number of threads).

built-in variables to determine the dimensions of the thread blocks and the grid on which the kernel was launched. This gives each thread a unique identity among all the threads running the kernel. This identity is frequently used to determine what data or operations a thread is responsible for.

All threads of a thread block are executed in a single SM. This allows threads within a thread block to communicate and synchronize with each other efficiently. Threads within a thread block all have access to the on-chip shared memory, which can be used for exchanging information between threads of a thread block.

A grid may consist of millions of thread blocks, while the GPU executing the grid may have only tens or hundreds of SMs. All threads of a thread block are executed by a single SM and, in most cases[1], run to completion on that SM. There is no guarantee of scheduling between thread blocks, so a thread block cannot rely on results from other thread blocks, as they may not be able to be scheduled until that thread block has completed. Figure 4 shows an example of how thread blocks from a grid are assigned to an SM.

The CUDA programming model enables arbitrarily large grids to run on GPUs of any size, whether it has only one SM or thousands of SMs. To achieve this, the CUDA programming model, with some exceptions, requires that there be no data dependencies between threads in different thread blocks. That is, a thread should not depend on results from or synchronize with a thread in a different thread block of the same grid. All the threads within a thread block run on the same SM at the same time. Different thread blocks within the grid are scheduled among the available SMs and may be executed in any order. In short, the CUDA programming model requires that it be possible to execute thread blocks in any order, in parallel or in series.

1.2.2.1.1 Thread Block Clusters

In addition to thread blocks, GPUs with compute capability 9.0 and higher have an optional level of grouping called *clusters*. Clusters are a group of thread blocks which, like thread blocks and grids, can be laid out in 1, 2, or 3 dimensions. Figure 5 illustrates a grid of thread blocks that is also organized into clusters. Specifying clusters does not change the grid dimensions or the indices of a thread block within a grid.

Specifying clusters groups adjacent thread blocks into clusters and provides some additional opportunities for synchronization and communication at the cluster level. Specifically, all thread blocks in a cluster are executed in a single GPC. Figure 6 shows how thread blocks are scheduled to SMs in a GPC when clusters are specified. Because the thread blocks are scheduled simultaneously and within a single GPC, threads in different blocks but within the same cluster can communicate and synchronize with each other using software interfaces provided by *Cooperative Groups*. Threads in clusters can access the shared memory of all blocks in the cluster, which is referred to as *distributed shared memory*.The maximum size of a cluster is hardware dependent and varies between devices.

Figure 6 illustrates the how thread blocks within a cluster are scheduled simultaneously on SMs within a GPC. Thread blocks within a cluster are always adjacent to each other within the grid.

1.2.2.2 Warps and SIMT

Within a thread block, threads are organized into groups of 32 threads called *warps*. A warp executes the kernel code in a *Single-Instruction Multiple-Threads* (SIMT) paradigm. In SIMT, all threads in the warp are executing the same kernel code, but each thread may follow different branches through the code. That is, though all threads of the program execute the same code, threads do not need to follow the same execution path.

[1] In certain situations when using features such as *CUDA Dynamic Parallelism*, a thread block may be suspended to memory. This means the state of the SM is stored to a system-managed area of GPU memory and the SM is freed to execute other thread blocks. This is similar to context swapping on CPUs. This is not common.

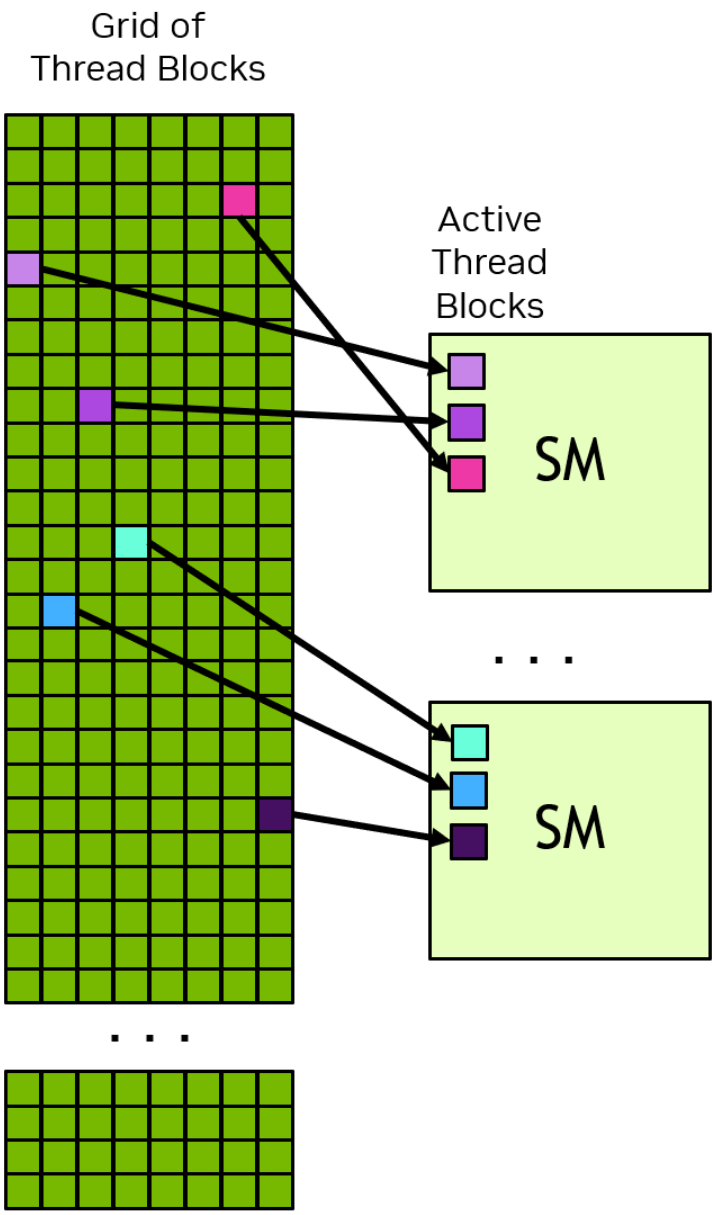

Figure 4: Each SM has one or more active thread blocks. In this example, each SM has three thread blocks scheduled simultaneously. There are no guarantees about the order in which thread blocks from a grid are assigned to SMs.

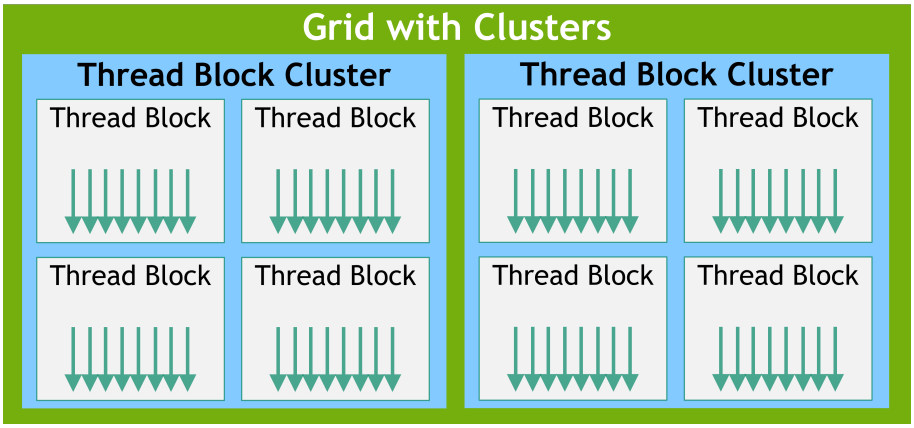

Figure 5: When clusters are specified, thread blocks are in the same location in the grid but also have a position within the containing cluster.

When threads are executed by a warp, they are assigned a warp lane. Warp lanes are numbered 0 to 31 and threads from a thread block are assigned to warps in a predictable fashion detailed in *Hardware Multithreading*.

All threads in the warp execute the same instruction simultaneously. If some threads within a warp follow a control flow branch in execution while others do not, the threads which do not follow the branch will be masked off while the threads which follow the branch are executed. For example, if a conditional is only true for half the threads in a warp, the other half of the warp would be masked off while the active threads execute those instructions. This situation is illustrated in Figure 7. When different threads in a warp follow different code paths, this is sometimes called warp divergence. It follows that utilization of the GPU is maximized when threads within a warp follow the same control flow path.

In the SIMT model, all threads in a warp progress through the kernel in lock step. Hardware execution may differ. See the sections on *Independent Thread Execution* for more information on where this distinction is important. Exploiting knowledge of how warp execution is actually mapped to real hardware is discouraged. The CUDA programming model and SIMT say that all threads in a warp progress through the code together. Hardware may optimize masked lanes in ways that are transparent to the program so long as the programming model is followed. If the program violates this model, this can result in undefined behavior that can be different in different GPU hardware.

While it is not necessary to consider warps when writing CUDA code, understanding the warp execution model is helpful in understanding concepts such as *global memory coalescing* and *shared memory bank access patterns*. Some advanced programming techniques use specialization of warps within a thread block to limit thread divergence and maximize utilization. This and other optimizations make use of the knowledge that threads are grouped into warps when executing.

One implication of warp execution is that thread blocks are best specified to have a total number of threads which is a multiple of 32. It is legal to use any number of threads, but when the total is not a multiple of 32, the last warp of the thread block will have some lanes that are unused throughout execution. This will likely lead to suboptimal functional units utilization and memory access for that warp.

> SIMT is often compared to Single Instruction Multiple Data (SIMD) parallelism, but there are some important differences. In SIMD, execution follows a single control flow path, while in SIMT, each thread is allowed to follow its own control flow path. Because of this, SIMT does not have a fixed data-width like SIMD. A more detailed discussion of SIMT can be found in *SIMT Execution Model*.

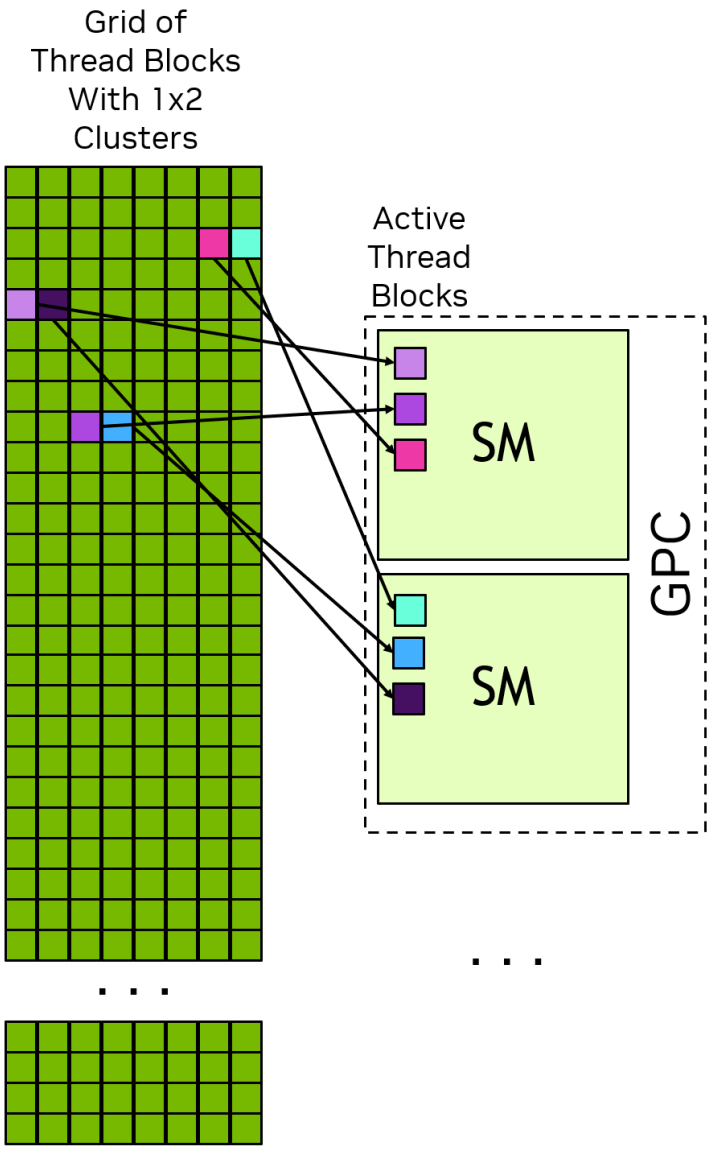

Figure 6: When clusters are specified, the thread blocks in a cluster are arranged in their cluster shape within the grid. The thread blocks of a cluster are scheduled simultaneously on the SMs of a single GPC.

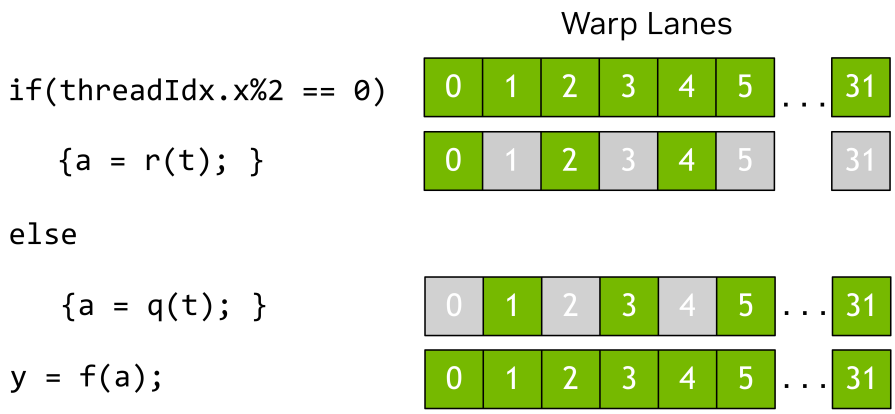

Figure 7: In this example, only threads with even thread index execute the body of the if statement, the others are masked off while the body is executed.

1.2.3. GPU Memory

In modern computing systems, efficiently utilizing memory is just as important as maximizing the use of functional units performing computations. Heterogeneous systems have multiple memory spaces, and GPUs contain various types of programmable on-chip memory in addition to caches. The following sections introduce these memory spaces in more details.

1.2.3.1 DRAM Memory in Heterogeneous Systems

GPUs and CPUs both have directly attached DRAM chips. In systems with more than one GPU, each GPU has its own memory. From the perspective of device code, the DRAM attached to the GPU is called *global memory*, because it is accessible to all SMs in the GPU. This terminology does not mean it is necessarily accessible everywhere within the system. The DRAM attached to the CPU(s) is called *system memory* or *host memory*.

Like CPUs, GPUs use virtual memory addressing. On all currently-supported systems, the CPU and GPU use a single unified virtual memory space. This means that the virtual memory address range for each GPU in the system is unique and distinct from the CPU and every other GPU in the system. For a given virtual memory address, it is possible to determine whether that address is in GPU memory or system memory and, on systems with multiple GPUs, which GPU memory contains that address.

There are CUDA APIs to allocate GPU memory, CPU memory, and to copy between allocations on the CPU and GPU, within a GPU, or between GPUs in multi-GPU systems. The locality of data can be explicitly controlled when desired. *Unified Memory*, discussed below, allows the placement of memory to be handled automatically by the CUDA runtime or system hardware.

1.2.3.2 On-Chip Memory in GPUs

In addition to the global memory, each GPU has some on-chip memory. Each SM has its own register file and shared memory. These memories are part of the SM and can be accessed extremely quickly from threads executing within the SM, but they are not accessible to threads running in other SMs.

The register file stores thread local variables which are usually allocated by the compiler. The shared memory is accessible by all threads within a thread block or cluster. Shared memory can be used for exchanging data between threads of a thread block or cluster.

The register file and unified data cache in an SM have finite sizes. The size of an SM's register file, unified data cache, and how the unified data cache can be configured for L1 and shared memory

balance can be found in *Memory Information per Compute Capability*. The register file, shared memory space, and L1 cache are shared among all threads in a thread block.

To schedule a thread block to an SM, the total number of registers needed for each thread multiplied by the number of threads in the thread block must be less than or equal to the available registers in the SM. If the number of registers required for a thread block exceeds the size of the register file, the kernel is not launchable and the number of threads in the thread block must be decreased to make the thread block launchable.

Shared memory allocations are done at the thread block level. That is, unlike register allocations which are per thread, allocations of shared memory are common to the entire thread block.

1.2.3.2.1 Caches

In addition to programmable memories, GPUs have both L1 and L2 caches. Each SM has an L1 cache which is part of the unified data cache. A larger L2 cache is shared by all SMs within a GPU. This can be seen in the GPU block diagram in Figure 2. Each SM also has a separate *constant cache*, which is used to cache values in global memory that have been declared to be constant over the life of a kernel. The compiler may place kernel parameters into constant memory as well. This can improve kernel performance by allowing kernel parameters to be cached in the SM separately from the L1 data cache.

1.2.3.3 Unified Memory

When an application allocates memory explicitly on the GPU or CPU, that memory is only accessible to code running on that device. That is, CPU memory can only be accessed from CPU code, and GPU memory can only be accessed from kernels running on the GPU[2] . CUDA APIs for copying memory between the CPU and GPU are used to explicitly copy data to the correct memory at the right time.

A CUDA feature called *unified memory* allows applications to make memory allocations which can be accessed from CPU or GPU. The CUDA runtime or underlying hardware enables access or relocates the data to the correct place when needed. Even with unified memory, optimal performance is attained by keeping the migration of memory to a minimum and accessing data from the processor directly attached to the memory where it resides as much as possible.

The hardware features of the system determine how access and exchange of data between memory spaces is achieved. Section *Unified Memory* introduces the different categories of unified memory systems. Section *Unified Memory* contains many more details about use and behavior of unified memory in all situations.

1.3. The CUDA platform

The NVIDIA CUDA platform consists of many pieces of software and hardware and many important technologies developed to enable computing on heterogeneous systems. This chapter serves to introduce some of the fundamental concepts and components of the CUDA platform that are important for application developers to understand. This chapter, like *Programming Model*, is not specific to any programming language, but applies to everything that uses the CUDA platform.

[2] An exception to this is *mapped memory*, which is CPU memory allocated with properties that enable it to be directly accessed from the GPU. However, mapped access occurs over the PCIe or NVLINK connection. The GPU is unable to hide the higher latency and lower bandwidth behind parallelism, so mapped memory is not a performant replacement to unified memory or placing data in the appropriate memory space.

1.3.1. Compute Capability and Streaming Multiprocessor Versions

Every NVIDIA GPU has a *Compute Capability* (CC) number, which indicates what features are supported by that GPU and specifies some hardware parameters for that GPU. These specifications are documented in the Section 5.1 appendix. A list of all NVIDIA GPUs and their compute capabilities is maintained on the CUDA GPU Compute Capability page.

Compute capability is denoted as a major and minor version number in the format X.Y where X is the major version number and Y is the minor version number. For example, CC 12.0 has a major version of 12 and a minor version of 0. The compute capability directly corresponds to the version number of the SM. For example, the SMs within a GPU of CC 12.0 have SM version *sm_120*. This version is used to label binaries.

Section 5.1.1 shows how to query and determine the compute capability of the GPU(s) in a system.

1.3.2. CUDA Toolkit and NVIDIA Driver

The *NVIDIA Driver* can be thought of as the operating system of the GPU. The NVIDIA Driver is a software component which must be installed on the host system's operating system and is necessary for all GPU uses, including display and graphical functionality. The NVIDIA Driver is foundational to the CUDA platform. In addition to CUDA, the NVIDIA Driver provides all other methods of using the GPU, for example Vulkan and Direct3D. The NVIDIA Driver has version numbers such as r580.

The *CUDA Toolkit* is a set of libraries, headers, and tools for writing, building, and analyzing software which utilizes GPU computing. The CUDA Toolkit is a separate software product from the NVIDIA driver

The *CUDA runtime* is a special case of one of the libraries provided by the CUDA Toolkit. The CUDA runtime provides both an API and some language extensions to handle common tasks such as allocating memory, copying data between GPUs and other GPUs or CPUs, and launching kernels. The API components of the CUDA runtime are referred to as the CUDA runtime API.

The CUDA Compatibility document provides full details of compatibility between different GPUs, NVIDIA Drivers, and CUDA Toolkit versions.

1.3.2.1 CUDA Runtime API and CUDA Driver API

The CUDA runtime API is implemented on top of a lower-level API called the *CUDA driver API*, which is an API exposed by the NVIDIA Driver. This guide focuses on the APIs exposed by the CUDA runtime API. All the same functionality can be achieved using only the driver API if desired. Some features are only available using the driver API. Applications may use either API or both interoperably. Section *The CUDA Driver API* covers interoperation between the runtime and driver APIs.

The full API reference for the CUDA runtime API functions can be found in the CUDA Runtime API Documentation .

The full API reference for the CUDA driver API can be found in the CUDA Driver API Documentation .

1.3.3. Parallel Thread Execution (PTX)

A fundamental but sometimes invisible layer of the CUDA platform is the *Parallel Thread Execution* (PTX) virtual instruction set architecture (ISA). PTX is a high-level assembly language for NVIDIA GPUs. PTX provides an abstraction layer over the physical ISA of real GPU hardware. Like other platforms,

applications can be written directly in this assembly language, though doing so can add unnecessary complexity and difficulty to software development.

Domain-specific languages and compilers for high-level languages can generate PTX code as an intermediate representation (IR) and then use NVIDIA's offline or just-in-time (JIT) compilation tools to produce executable binary GPU code. This enables the CUDA platform to be programmable from languages other than just those supported by NVIDIA-provided tools such as *NVCC: The NVIDIA CUDA Compiler*.

Since GPU capabilities change and grow over time, the PTX virtual ISA specification is versioned. PTX versions, like SM versions, correspond to a compute capability. For example, PTX which supports all the features of compute capability 8.0 is called *compute_80*.

Full documentation on PTX can be found in the PTX ISA .

1.3.4. Cubins and Fatbins

CUDA applications and libraries are usually written in a higher-level language like C++. That higher-level language is compiled to PTX, and then the PTX is compiled into real binary for a physical GPU, called a *CUDA binary*, or *cubin* for short. A cubin has a specific binary format for a specific SM version, such as *sm_120*.

Executables and library binaries that use GPU computing contain both CPU and GPU code. The GPU code is stored within a container called a *fatbin*. Fatbins can contain cubins and PTX for multiple different targets. For example, an application could be built with binaries for multiple different GPU architectures, that is, different SM versions. When an application is run, its GPU code is loaded onto a specific GPU and the best binary for that GPU from the fatbin is used.

Fatbins can also contain one or more PTX versions of GPU code, the use for which is described in *PTX Compatibility*. Figure 8 shows an example of an application or library binary which contains multiple cubin versions of GPU code as well as one version of PTX code.

1.3.4.1 Binary Compatibility

NVIDIA GPUs guarantee binary compatibility in certain circumstances. Specifically, within a major version of compute capability, GPUs with minor compute capability greater than or equal to the targeted version of cubin can load and execute that cubin. For example, if an application contains a cubin with code compiled for compute capability 8.6, that cubin can be loaded and executed on GPUs with compute capability 8.6 or 8.9. It cannot, however, be loaded on GPUs with compute capability 8.0, because the GPU's CC minor version, 0, is lower than the code's minor version, 6.

NVIDIA GPUs are not binary compatible between major compute capability versions. That is, cubin code compiled for compute capability 8.6 will not load on GPUs of compute capability 9.0.

When discussing binary code, the binary code is often referred to as having a version such as *sm_86* in the above example. This is the same as saying the binary was built for compute capability 8.6. This shorthand is often used because it is how a developer specifies this binary build target to the NVIDIA CUDA compiler, *nvcc*.

Note

Binary compatibility is promised only for binaries created by NVIDIA tools such as `nvcc`. Manual editing or generating binary code for NVIDIA GPUs is not supported. Compatibility promises are invalidated if binaries are modified in any way.

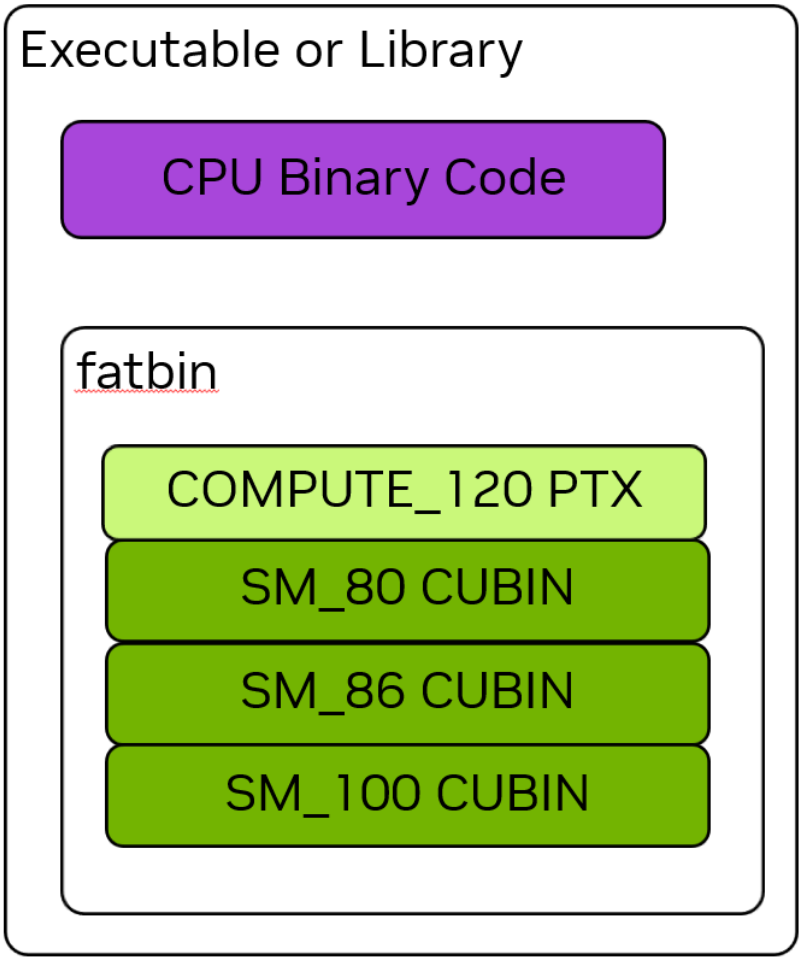

Figure 8: The binary for an executable or library contains both CPU binary code and a fatbin container for GPU code. A fatbin can contain both cubin GPU binary code and PTX virtual ISA code. PTX code can be JIT compiled for future targets.

1.3.4.2 PTX Compatibility

GPU code can be stored in executables in binary or PTX form, which is covered in *Cubins and Fatbins*. When an application stores the PTX version of GPU code, that PTX can be JIT compiled at application runtime for any compute capability equal or higher to the compute capability of the PTX code. For example, if an application contains PTX for *compute_80*, that PTX code can be JIT compiled to later SM versions, such as *sm_120* at application runtime. This enables forward compatibility with future GPUs without the need to rebuild applications or libraries.

1.3.4.3 Just-in-Time Compilation

PTX code loaded by an application at runtime is compiled to binary code by the device driver. This is called just-in-time (JIT) compilation. Just-in-time compilation increases application load time, but allows the application to benefit from any new compiler improvements coming with each new device driver. It also enables applications to run on devices that did not exist at the time the application was compiled.

When the device driver just-in-time compiles PTX code for an application, it automatically caches a copy of the generated binary code in order to avoid repeating the compilation in subsequent invocations of the application. The cache - called the compute cache - is automatically invalidated when the device driver is upgraded, so that applications can benefit from the improvements in the new just-in-time compiler built into the device driver.

How and when PTX is JIT compiled at runtime has been relaxed since the earliest versions of CUDA, allowing more flexibility for when and if to JIT compile some or all kernels. The section *Lazy Loading* describes the available options and how to control JIT behavior. There are also a few environment variables which control just-in-time compilation behavior, as described in *CUDA Environment Variables*.

As an alternative to using `nvcc` to compile CUDA C++ device code, NVRTC can be used to compile CUDA C++ device code to PTX at runtime. NVRTC is a runtime compilation library for CUDA C++; more information can be found in the NVRTC User guide.